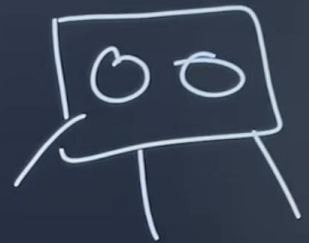
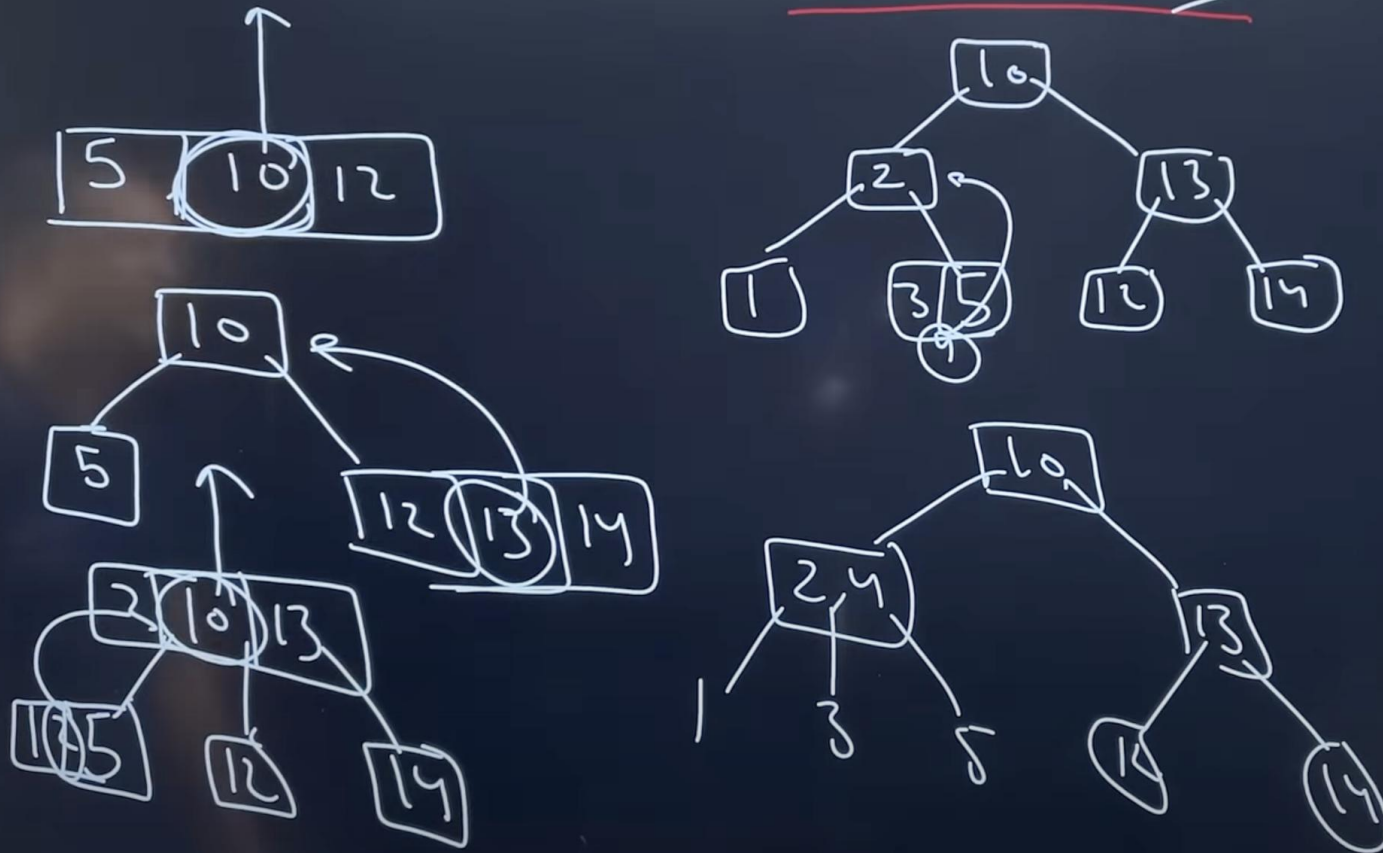


Q Consider the following elements ~~5, 10, 12, 13, 14, 1, 2, 3, 4~~
insert them into an empty b-tree of order = 3.



Insertion in B-TREE

- A B-tree starts with a single root node (which is also a leaf node) at level 0 (zero). Once the root node is full with $m - 1$ search key values and we attempt to insert another entry in the tree, the root node splits into two nodes at level 1.
- Only the middle value is kept in the root node, and the rest of the values are split evenly between the two new nodes. When a non-root node is full and a new entry is inserted into it, that node is split into two new nodes at the same level, and the middle entry is moved to the parent node along with two pointers to the new nodes.
- If the parent node is full, it is also split. Splitting can propagate all the way to the root node, creating a new root node. If the root is split, the root is split.



2:45:27 / 5:33:30 • (Chapter-5: Indexing)- Overview of indexing, Primary indexing, Clustered indexing ... >



B tree

- A B-tree of order m if non-empty is an m -way search tree in which.
 - The root has at least zero child nodes and at most m child nodes.
 - The internal nodes except the root have at least $\lceil m/2 \rceil$ child nodes and at most m child nodes.
 - The number of keys in each internal node is one less than the number of child nodes and these keys partition the subtrees of the nodes in a manner similar to that of m -way search tree.
 - All leaf nodes are on the same level(perfectly balanced).

Root

Rules	MAX	MIN
CHILD	m	0
DATA	$m-1$	1

Internal except Root

Rules	MAX	MIN
CHILD	m	$\lceil m/2 \rceil$
DATA	$m-1$	$\lceil m/2 \rceil - 1$

Leaf

Rules	MAX	MIN
CHILD	0	0
DATA	$m-1$	$\lceil m/2 \rceil - 1$

Dense Vs Sparse

- **Dense Index** In dense index, there is an entry in the index file for every search key value in the main file. This makes searching faster but requires more space to store index records itself. Note that it is not for every record, it is for every search key value.
 - ◉ Sometime number of records in the main file $>$ number of search keys in the main file for example if search key is repeated.
- **Sparse Index**-If an index entry is created only for some records of the main file, then it is called sparse index. No. of index entries in the index file $<$ No. of records in the main file. Note: - dense and sparse are not complementary to each other, sometimes it is possible that a record is both dense and sparse.

View chapter

(Chapter-5: Indexing)- Overview of indexing, Primary indexing, Clustered indexing and Secondary Indexing, B-Tree.

SUBSCRIBE
KG

SECONDARY INDEXING

- Most common scenarios, suppose that we already have a primary indexing on primary key, but there is frequent query on some other attributes, so we may decide to have one more index file with some other attribute.
- Main file is ordered according to the attribute on which indexing is done(unordered).
- Secondary indexing can be done on key or non-key attribute.
- No of entries in the index file is same as the number of entries in the main file.
- It is an example of dense indexing.

CLUSTERED INDEXING

- Main file will be ordered on some non-key attributes
- No of entries in the index file = no of unique values of the attribute on which indexing is done.
- ▶ It is the example of Sparse as well as dense indexing



PRIMARY INDEXING

- Main file is always sorted according to primary key.
- Indexing is done on Primary Key, therefore called as primary indexing
- Index file have two columns, first primary key and second anchor pointer (base address of block)
- It is an example of Sparse Indexing.
- Here first record (anchor record) of every block gets an entry in the index file
- No. of entries in the index file = No of blocks acquired by the main file.

Subtitles/closed captions (c)

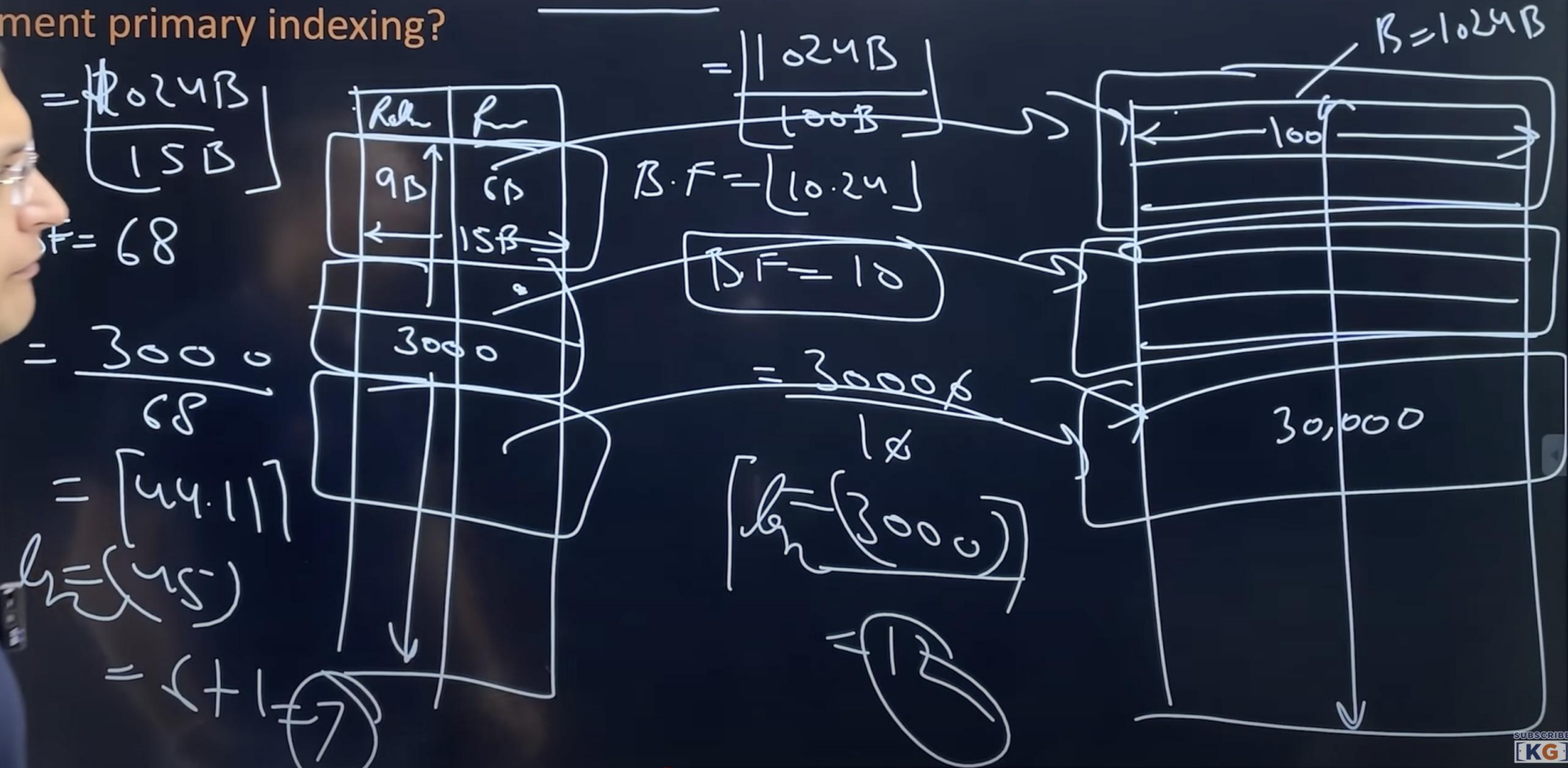


2:37:10 / 5:33:30 • (Chapter-5: Indexing)- Overview of indexing, Primary indexing, Clustered indexing ... >



1. Indexes can be established on any relation field, be it primary key or non-key.
 2. Each attribute can have a dedicated index file, meaning multiple index files may exist for one main file.
 3. Index files are always organized, allowing for the utilization of binary search advantages, irrespective of the main file's order.
 4. Indexing accelerates data retrieval time but also introduces space overhead for storing the index file.
- The correct block in the main file can be located with $\log_2(\text{number of blocks in index file}) + 1$ accesses.

Q. Suppose we have ordered file with records stored $r = 30,000$ on a disk with Block Size $B = 1024$ B. File records are of fixed size and are unspanned with record length $R = 100$ B. Suppose that ordering key field of file is 9 B long and a block pointer is 6 B long, implement primary indexing?



Unordered file organization

- Records are typically added at the end of the file, without following any specific order.
- This insertion method allows only linear search, resulting in slower search times.
- Despite slow searches, maintenance including insertion and deletion is simpler.
- No reorganization of the entire file is needed, making maintenance easier.
- If file is unordered then no of block accesses required to reach correct block which contain the desired record is $O(n)$, where n is the number of blocks.

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

File organization/ organization of records in a fileOrdered file organization

- All the records in the file are ordered on some search key field.
- Here binary search is possible. (give example of book page searching)
- Maintenance (insertion & deletion) is costly, as it requires reorganization of entire file.
- Notes that we will get binary search only if we are using that key for searching on which indexing is done, otherwise it will behave as unsorted file.
- if file is unordered then no of block accesses required to reach correct block which contain the desired record is $O(\log_2 n)$, where n is the number of blocks.

```

-rw-r----- 1 mysql mysql 10M ul 16 20:10 node__field_meta_tags.ibd
-rw-r----- 1 mysql mysql 10M ul 15 22:50 comment_field_data.ibd
-rw-r----- 1 mysql mysql 11M ul 16 20:18 search_total.ibd
-rw-r----- 1 mysql mysql 11M ul 16 20:21 cache_discovery.ibd
-rw-r----- 1 mysql mysql 12M ul 16 20:11 node_field_data.ibd
-rw-r----- 1 mysql mysql 14M ul 16 20:11 url_alias.ibd
-rw-r----- 1 mysql mysql 14M ul 16 20:11 taxonomy_index.ibd
-rw-r----- 1 mysql mysql 15M ul 16 19:27 node__tags.ibd
-rw-r----- 1 mysql mysql 15M ul 15 22:47 comment__comment_body.ibd
-rw-r----- 1 mysql mysql 16M ul 16 19:27 node_revision__tags.ibd
-rw-r----- 1 mysql mysql 19M ul 16 20:23 sessions.ibd
-rw-r----- 1 mysql mysql 22M ul 16 20:18 search_dataset.ibd
-rw-r----- 1 mysql mysql 31M ul 16 20:11 node__body.ibd
-rw-r----- 1 mysql mysql 32M ul 16 20:24 watchdog.ibd
-rw-r----- 1 mysql mysql 36M ul 16 20:11 node_revision__body.ibd
-rw-r----- 1 mysql mysql 108M ul 16 20:18 search_index.ibd
-rw-r----- 1 mysql mysql 120M ul 16 20:21 cache_entity.ibd
-rw-r----- 1 mysql mysql 268M ul 16 20:25 cache_data.ibd
-rw-r----- 1 mysql mysql 1.6G ul 16 20:25 cache_dynamic_page_cache.ibd
-rw-r----- 1 mysql mysql 3.4G ul 16 20:25 cache_render.ibd

```


Indexing

Relational databases are based on set theory.

- In set theory, the order of elements is unimportant, similarly in database tables.
- However, in practical implementation, element order in tables is often specified.
- Various operations such as search, insertion, and deletion are influenced by the order of elements in the tables.
- Elements in a table can be stored in two ways: sorted (ordered) or unsorted (unordered).

